

Database Management Systems



Shamim Ahmad

Transaction

The background features a dark blue field with several parallel diagonal stripes in a slightly lighter shade of blue. A single, solid light blue horizontal line is positioned on the left side, extending towards the center.

TRANSACTION EXAMPLE:

- Suppose, Mr. A and Mr. Y have a bank account. Now need to transfer 1000 taka from Mr. A accounts to Mr. Y accounts.

- Transaction:

Read (Mr.A)

$\text{Mr.A} = \text{Mr.A} - 1000$

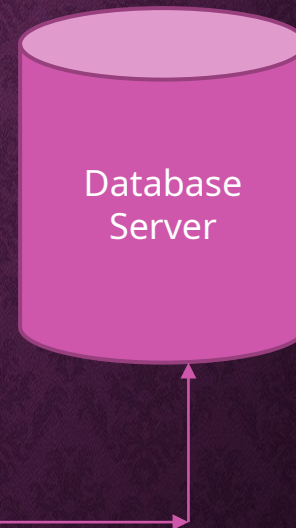
Write (Mr.A)

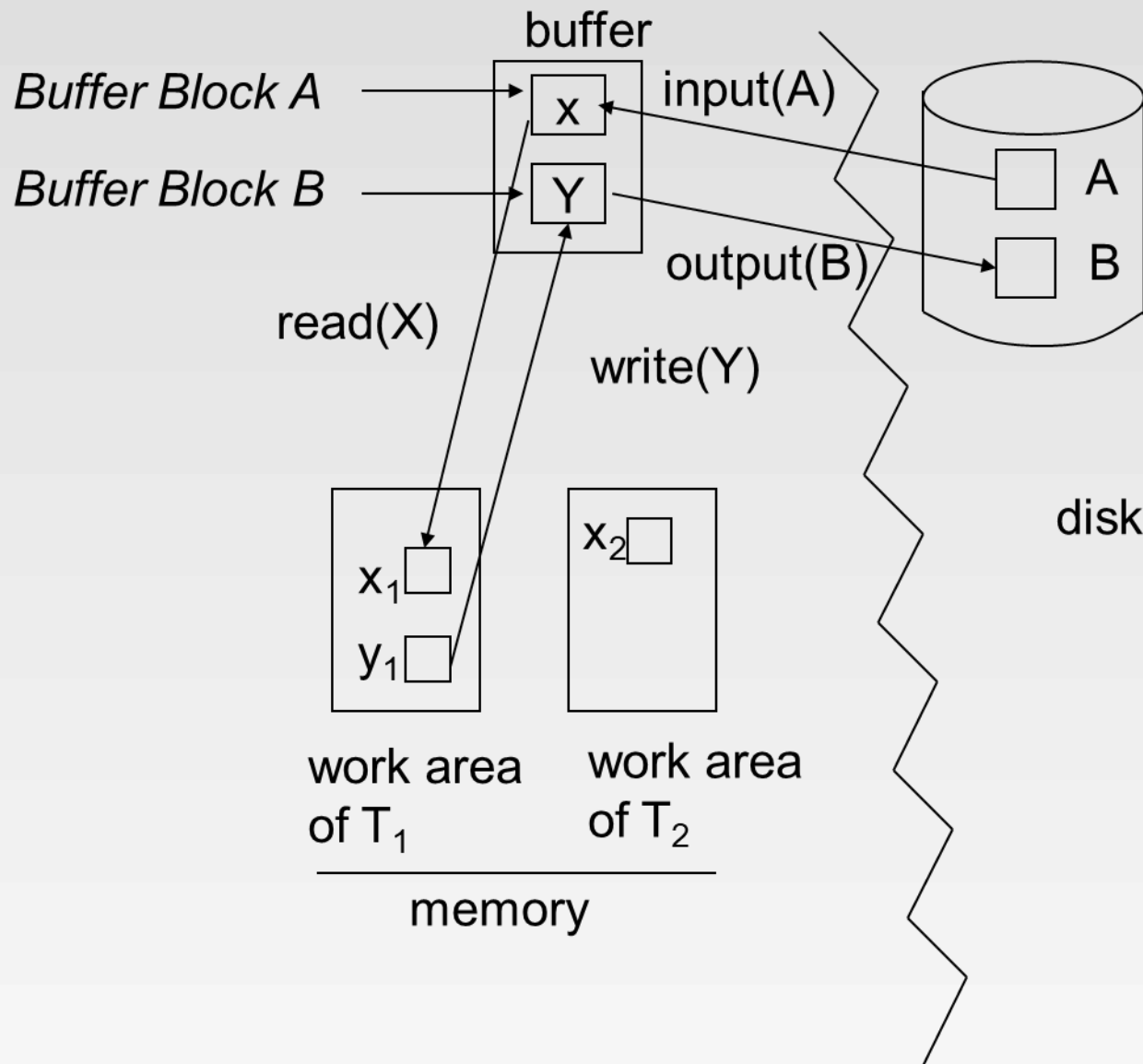
Read (Mr.Y)

$\text{Mr. Y} = \text{Mr.Y} + 1000$

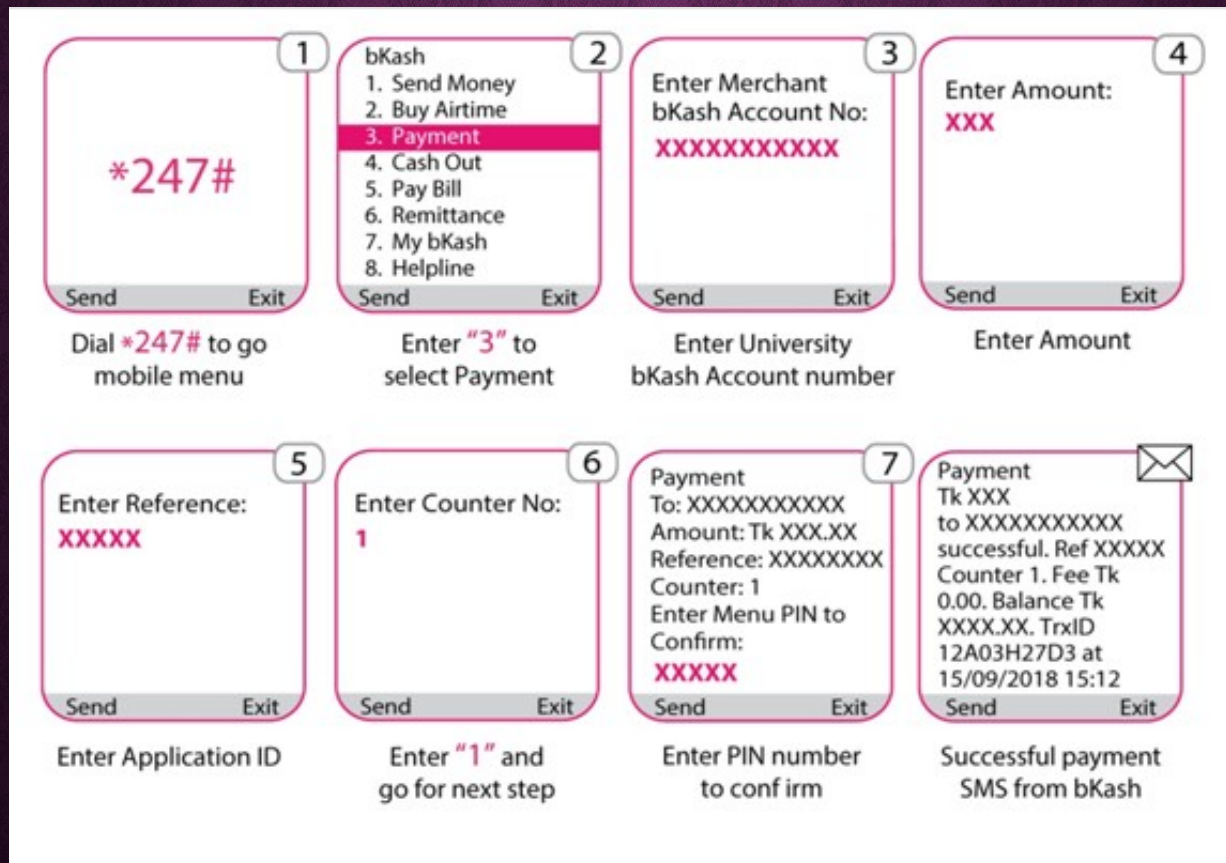
Write (Mr.Y)

COMMIT





TRANSACTION EXAMPLE (CONT'D):

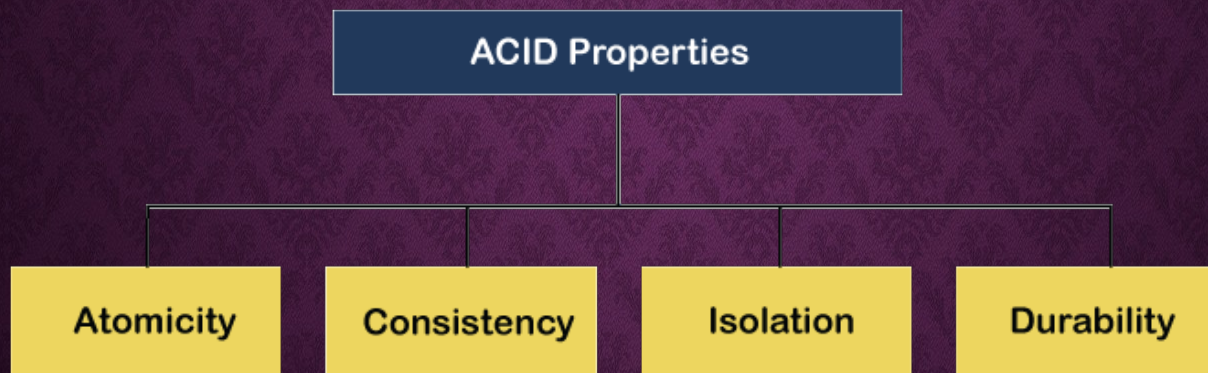


Transaction Concept

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
- E.g. transaction to transfer €50 from account A to account B:
 1. **read_from_account**(A)
 2. $A := A - 50$
 3. **write_to_account**(A)
 4. **read_from_account**(B)
 5. $B := B + 50$
 6. **write_to_account**(B)
- Two main issues to deal with:
 - Failures of various kinds, such as **hardware failures and system crashes**
 - Concurrent execution of **multiple transactions**

ACID PROPERTIES

- In order to maintain consistency in a database, before and after the transaction, certain properties are followed. These are called **ACID** properties.



Transaction ACID properties

- E.g. transaction to transfer €50 from account A to account B:

1. **read_from_account**(A)
2. $A := A - 50$
3. **write_to_account**(A)
4. **read_from_account**(B)
5. $B := B + 50$
6. **write_to_account**(B)

- **Atomicity requirement**

- if the transaction fails **after step 3 and before step 6**, money will be “lost” leading to an inconsistent database state
 - ▶ Failure could be due to **software or hardware**
- the system should ensure that **updates of a partially executed** transaction are not reflected in the database
- **All or nothing**, regarding the execution of the transaction

- **Durability requirement** — once the user has been notified **of transaction has completion**, the **updates must persist** in the database even if there are **software or hardware failures**.

Transaction ACID properties (Cont.)

- Transaction to transfer €50 from account A to account B:
 1. **read_from_account**(A)
 2. $A := A - 50$
 3. **write_to_account**(A)
 4. **read_from_account**(B)
 5. $B := B + 50$
 6. **write_to_account**(B)
- **Consistency requirement** in above example:
 - **the sum of A and B is unchanged by the execution of the transaction**
- In general, consistency requirements include
 - ▶ Explicitly **specified integrity constraints** such as primary keys and foreign keys
 - ▶ Implicit integrity constraints
 - e.g. **sum of balances of all accounts**, minus sum of loan amounts must equal value of cash-in-hand
 - A transaction must see a consistent database and must leave a consistent database
 - During transaction execution the database may be **temporarily inconsistent**.
 - ▶ Constraints to be **verified** only at the **end of the transaction**

Transaction ACID properties (Cont.)

- **Isolation requirement** — if between **steps 3 and 6**, **another transaction T2 is allowed** to access the **partially updated database**, it will see an inconsistent database (**the sum $A + B$ will be less** than it should be).

T1

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

T2

read(A), read(B), print($A+B$)

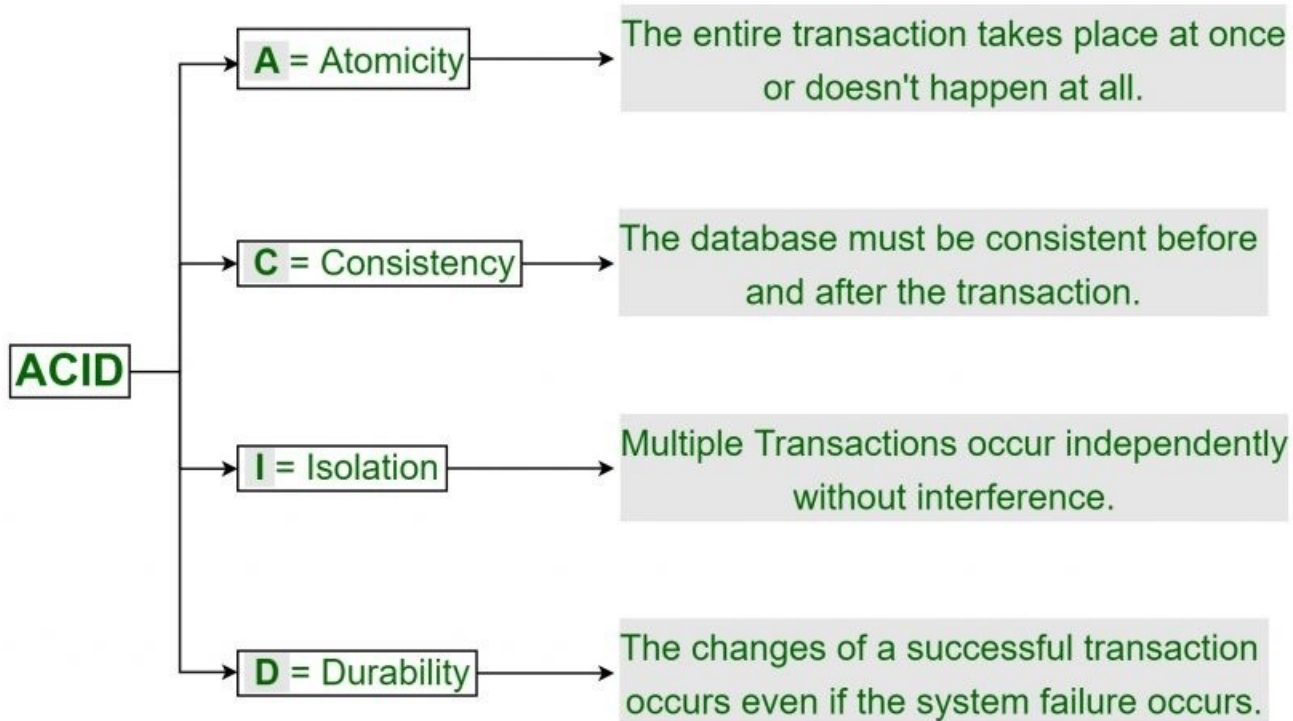
- Isolation can be ensured trivially by running transactions **serially**
 - that is, **one after the other**.
- However, executing multiple **transactions concurrently** has significant benefits, as we will see later.

ACID Properties - Summary

A **transaction** is a **unit of program execution** that accesses and possibly updates various data items. To preserve the **integrity of data** the database system must ensure:

- **Atomicity** Either **all operations** of the transaction are properly reflected in the database or **none are**.
- **Consistency** Execution of a (single) transaction preserves the **consistency of the database**.
- **Isolation** Although multiple transactions may **execute concurrently**, each transaction must be unaware of other concurrently executing transactions. **Intermediate transaction results** must be hidden from other concurrently executed transactions.
 - That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j **finished** execution before T_i **started**, or T_j **started** execution after T_i **finished**.
- **Durability**. After a transaction completes successfully, the changes it has made to the **database persist**, even if there are **system failures**.

ACID Properties in DBMS

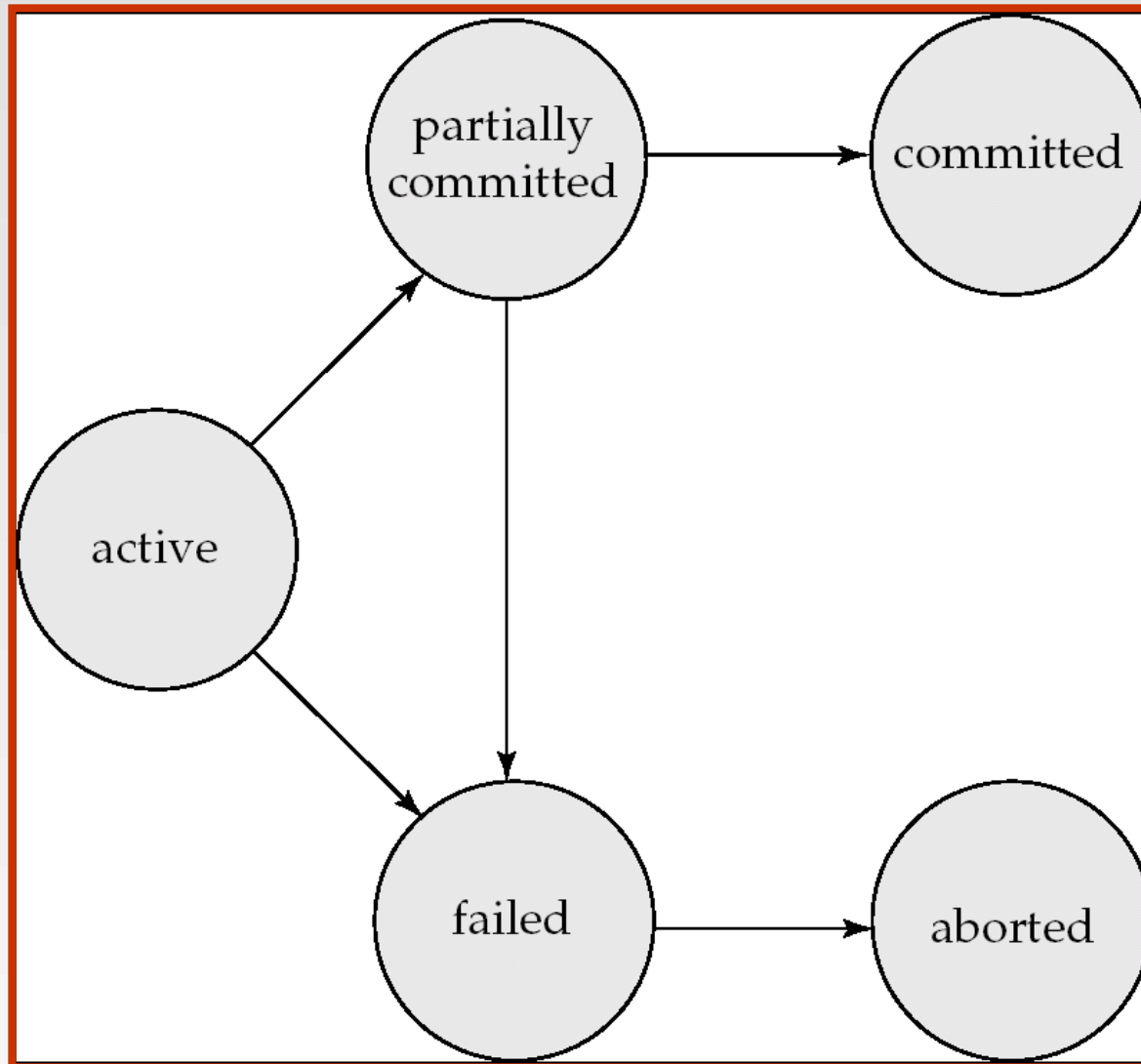


Non-ACID Transactions

- There are application domains where ACID properties are not **necessarily desired or, most likely, not always possible.**
- This is the case of so-called **long-duration transactions**
 - Suppose that a transaction takes a lot of time
 - In this case it is **unlikely** that isolation can/should be guaranteed
 - ▶ E.g. Consider a transaction of **booking a hotel and a flight**
- Without **Isolation**, **Atomicity may be compromised**
- **Consistency and Durability should be preserved**
- Usual solution for long-duration transaction is to define **compensation action** – **what to do if later the transaction fails**
- In (centralized) databases long-duration transactions are usually not considered.
- But these are more and more important, specially in the **context of the Web.**

	A	C	I	D
read_from_acoount(A)				
$A := A - 50$				
write_to_account(A)				
read_from_accont(B)				
$B := B + 50$				
write_to_account(B)				

Transaction State (Cont.)



Transaction State

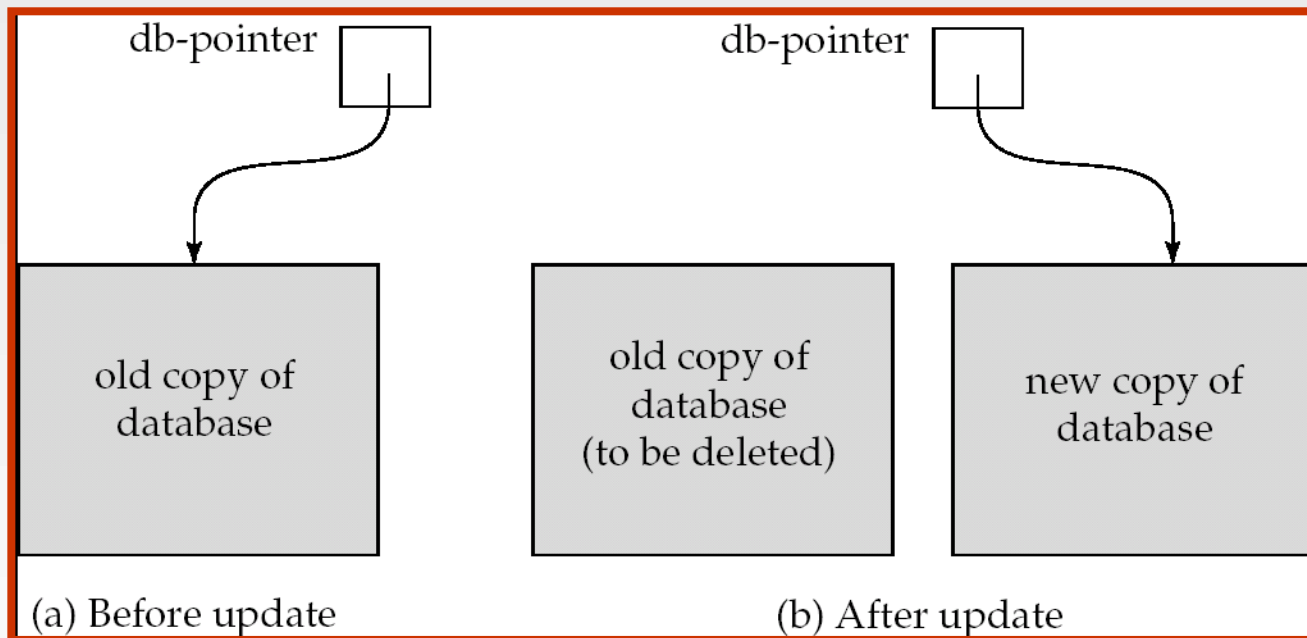
- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the **final statement** has been executed.
- **Failed** – after the discovery that **normal execution** can **no longer proceed**.
- **Aborted** – after the transaction has been **rolled back** and the database **restored to its state prior** to the start of the transaction. Two options after it has been aborted:
 - **restart** the transaction
 - ▶ can be done only if no internal logical error
 - **kill** the transaction
- **Committed** – after **successful completion**.
- To guarantee atomicity, **external observable action** should all be performed (in order) after the **transaction is committed**.

Committed Transactions

- A committed transaction that has performed updates transforms the database into anew **consistent state**, which **must persist** even if there is a **system failure**.
- It cannot undo its effects by aborting it
- The only way to undo the effects of a committed transaction is to execute a **compensating transaction**.

Implementation of Atomicity and Durability

- The **recovery-management** component of a database system implements the support for atomicity and durability.
- E.g. the **shadow-database** scheme:
 - all updates are made on a **shadow copy** of the database
 - ▶ **db_pointer** is made to point to the updated shadow copy after
 - the transaction **reaches partial commit** and
 - all updated pages have been flushed to disk.



Implementation of Atomicity and Durability

- The **recovery-management** component of a database system implements the support for atomicity and durability.
- E.g. the **shadow-database** scheme:
 - all updates are made on a **shadow copy** of the database
 - ▶ **db_pointer** is made to point to the updated shadow copy after
 - the transaction reaches partial commit and
 - after the operating system has written all the pages to disk

